Table of Contents

Blueprint Arsitektur Enterprise 1
Retail Management System (RMS) — Multi-Toko 1
1. Ringkasan Eksekutif 1
2. Prinsip Arsitektur 2
3. Tech Stack 2
4. Arsitektur Modul (Bounded Context) & Ruang Lingkup Fungsional 5
5. Entity Relationship Diagram (ERD) — Skema Database Inti 6
6. Use Case Diagram 10
7. Class Diagram (Domain Model Inti) 11
8. Sequence Diagram — Alur Kritis 14
9. State Diagram — Siklus Hidup Pesanan (Order) 16
10. Component & Deployment Diagram 17
11. Ringkasan API (Contoh Endpoint per Modul) 18
12. Strategi PWA (Progressive Web App) 18
13. Keamanan & Kepatuhan 19
14. Roadmap Implementasi 19
15. Pertimbangan Skalabilitas Masa Depan 19

Blueprint Arsitektur Enterprise

Retail Management System (RMS) — Multi-Toko

Disusun sebagai: Enterprise Architecture Blueprint **Cakupan:** Pemesanan, Kasir & Pembayaran, Supplier & Pembelian, Stok, Keuangan, Barang Masuk, Dashboard Analitik
Target Platform: Web Responsive + PWA (mobile-first, offline-capable)

---

1. Ringkasan Eksekutif

RMS adalah sistem retail terintegrasi yang menghubungkan siklus bisnis end-to-end: **Supplier → Pembelian → Barang Masuk → Stok → Penjualan (Kasir/Online) → Keuangan → Analitik**. Pendekatan arsitektur menggunakan **Modular Monolith berbasis Domain-Driven Design (DDD)** pada fase awal (MVP–Growth), dengan batas modul

(bounded context) yang jelas sehingga siap dipecah menjadi microservices saat skala bisnis membutuhkan (multi-cabang > 20 toko, transaksi > 1 juta/bulan).

Alasan memilih Modular Monolith terlebih dahulu (bukan microservices langsung): - Kompleksitas operasional lebih rendah, cocok untuk tim kecil-menengah (5–15 engineer) - Konsistensi transaksi lintas modul (stok ↔ keuangan ↔ penjualan) lebih mudah dijaga (ACID) dibanding eventual consistency ala microservices - Biaya infrastruktur lebih efisien di tahap awal - Migrasi ke microservices tetap mudah karena setiap modul sudah punya boundary domain yang tegas

---

2. Prinsip Arsitektur

Prinsip	Penerapan
Domain-Driven Design	Setiap modul = bounded context dengan model domain sendiri
API-First	Semua modul berkomunikasi lewat REST/OpenAPI & event, tidak ada shared DB table lintas modul
Event-Driven untuk konsistensi lintas modul	Perubahan stok, penjualan, pembelian memicu event ke modul Keuangan (double-entry otomatis)
Offline-first di titik kritis	Aplikasi Kasir wajib tetap bisa transaksi saat internet putus (PWA + local queue)
Security by design	RBAC + audit trail di semua modul finansial
CQRS ringan untuk Dashboard	Query analitik dipisahkan dari OLTP (memakai tabel agregat/summary) agar tidak membebani transaksi kasir

---

3. Tech Stack

3.1 Frontend Web & PWA

Layer	Teknologi	Alasan
Framework	**HTML5 semantik + JavaScript ES2022 (ES Modules, tanpa framework)**	Halaman katalog dirender dari template HTML oleh server Python (SEO pemesanan online), kasir & dashboard berupa halaman JS (SPA ringan)
Bahasa	**JavaScript (ES2022) + JSDoc**	Anotasi tipe lewat JSDoc, tanpa langkah kompilasi
Styling/UI	**CSS3 murni (CSS Variables, Flexbox, Grid, Media Query)**	Design token konsisten, mobile-first, tanpa dependensi build
State Management	**Fetch API + cache sederhana (Map / Cache Storage)** (server state) + **objek state JavaScript**	Ringan, tanpa library tambahan

Layer	Teknologi	Alasan
	dengan pola pub/sub (client/local state)	
PWA Engine	**Service Worker native** + **Web App Manifest**	Installable, caching strategy per modul ditulis manual di sw.js
Offline Storage	**IndexedDB API native (JavaScript)**	Antrian transaksi kasir saat offline
Realtime	**WebSocket API native (browser)**	Update stok, notifikasi order real-time ke dashboard/kasir
Formulir & Validasi	**HTML5 Constraint Validation + validasi JavaScript**	Validasi di browser, divalidasi ulang di server Python (Pydantic)
Chart/Visualisasi	**Chart.js (library JavaScript)**	Dashboard analitik
Barcode/Scanner	**BarcodeDetector API native / html5-qrcode (JavaScript)**	Scan produk via kamera device kasir
Print Struk	**WebUSB/WebSerial API + perintah ESC/POS ditulis dalam JavaScript** atau print via browser ke printer thermal jaringan	Cetak struk kasir

3.2 Backend

Layer	Teknologi	Alasan
Framework	**Python 3.12 + FastAPI**	Modular lewat APIRouter per bounded context (cocok DDD), dependency injection bawaan, async, mudah dipecah jadi microservices
API Style	**REST (OpenAPI 3.0)** + **WebSocket Gateway** (FastAPI WebSocket endpoint)	REST untuk CRUD/transaksi, WS untuk realtime dashboard & stok
ORM	**SQLAlchemy 2.0 + Alembic**	ORM matang untuk MySQL (driver PyMySQL, URL mysql+pymysql://), migrasi skema terkelola
Validasi	**Pydantic v2**	Validasi skema request/response otomatis, menghasilkan OpenAPI
Antrian/Event Bus	**Transactional Outbox di MySQL (tabel event_outbox) + Celery (broker SQLAlchemy/MySQL)** (RabbitMQ/Kafka jika skala besar)	Event: `stock.reduced`, `purchase.received`, `sale.completed`, `payment.confirmed` → dikonsumsi modul Keuangan & Stok

Layer	Teknologi	Alasan
Job Scheduler	**Celery + Celery Beat (broker: MySQL via SQLAlchemy transport)**	Rekonsiliasi stok harian, laporan berkala, retry pembayaran
Auth & Otorisasi	**FastAPI OAuth2 + PyJWT (passlib/argon2 untuk hash password)** dengan **JWT + Refresh Token** serta **RBAC/ABAC**	Multi-role: Owner, Admin, Kasir, Gudang, Finance
Payment Gateway	**Midtrans / Xendit** (VA, QRIS, e-wallet, kartu)	Standar payment Indonesia

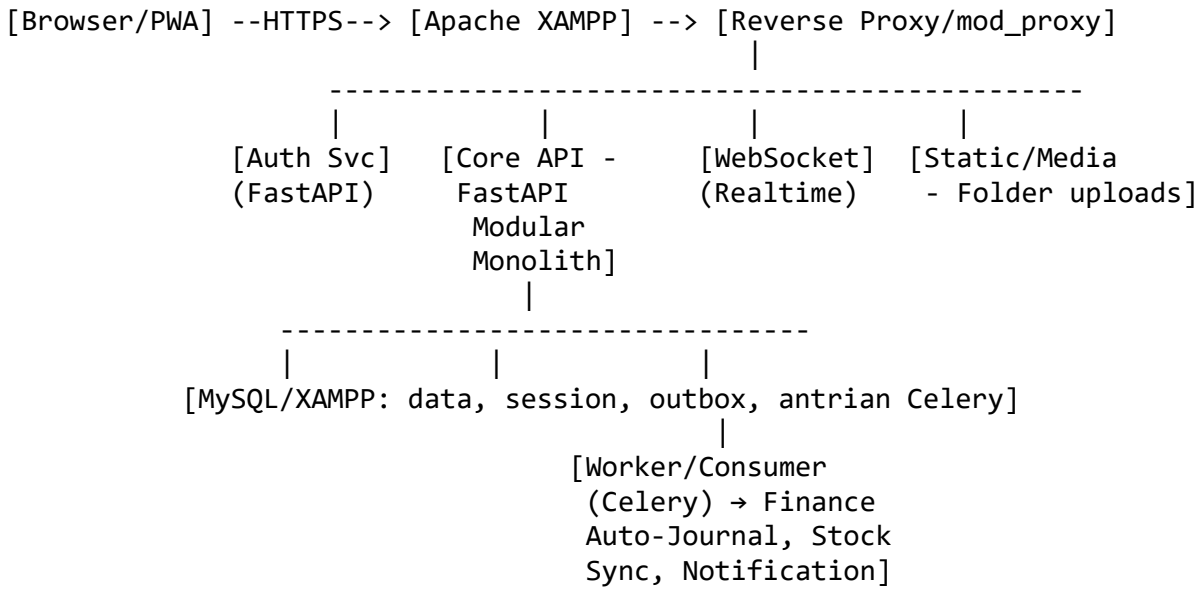
3.3 Data Layer

Kebutuhan	Teknologi
Database transaksional utama (OLTP)	**MySQL (MariaDB bawaan XAMPP), engine InnoDB, dikelola lewat phpMyAdmin**
Cache, session, rate-limit	**Tabel MySQL (session) + cache in-memory Python (cachetools); rate-limit dengan slowapi**
Data warehouse untuk analitik berat	**Tabel agregat terjadwal di MySQL (Celery Beat)** atau **MySQL read-replica terpisah** (opsional, fase scale-up)
Full-text search produk	**MySQL FULLTEXT index** (awal) → **Elasticsearch/Meilisearch** (skala besar)
File/objek (foto produk, invoice, lampiran)	**Folder lokal server (uploads) yang disajikan Apache; path file disimpan di MySQL**

3.4 Infrastruktur & DevOps

Layer	Teknologi
Web Server & Runtime	**XAMPP (Apache + MySQL/MariaDB + phpMyAdmin) dan Python virtualenv (venv)**
Proses Manager	**Uvicorn/Gunicorn (FastAPI) dan Celery worker dijalankan sebagai service (NSSM di Windows / systemd di Linux)**
Reverse Proxy	**Apache XAMPP (mod_proxy, mod_proxy_wstunnel untuk WebSocket)** dengan TLS (Let's Encrypt)
CI/CD	**GitHub Actions** → lint, test (pytest), deploy via SSH/rsync
IaC	**Skrip setup (PowerShell/Bash) + requirements.txt**
Monitoring Metrik	**Endpoint /health FastAPI + MySQL slow query log**
Logging Terpusat	**Modul logging Python (file log rotasi)** atau **Apache access/error log**
Error Tracking	**Sentry**
API Gateway (opsional, saat microservices)	**Apache XAMPP (mod_proxy)**

3.5 Ringkasan Arsitektur Fisik



---

4. Arsitektur Modul (Bounded Context) & Ruang Lingkup Fungsional

Modul 1 — Pemesanan Produk oleh Pelanggan (Ordering)

- Katalog produk (kategori, varian, harga, stok real-time)
- Keranjang belanja, checkout, pilih metode pengiriman/ambil di toko
- Riwayat pesanan, status pesanan (pending → diproses → dikirim/siap diambil → selesai)
- Notifikasi status via WebSocket/Push Notification (PWA)
- Integrasi ke modul Kasir/Pembayaran & Stok

Modul 2 — Aplikasi Kasir & Pembayaran (POS)

- Input transaksi manual/scan barcode
- Diskon, promo, member/loyalty point
- Split payment (tunai, QRIS, kartu, e-wallet) via payment gateway
- **Mode offline**: transaksi tetap tercatat lokal (IndexedDB), sinkron otomatis saat online kembali
- Cetak/kirim struk digital
- Rekonsiliasi kas per shift (buka/tutup kasir)

Modul 3 — Manajemen Supplier & Pembelian (Procurement)

- Master data supplier (kontak, termin pembayaran, histori)
- Purchase Order (PO): draft → approval → dikirim ke supplier
- Tracking status PO, revisi, pembatalan
- Perbandingan harga antar supplier (opsional)

Modul 4 — Manajemen Stok & Persediaan (Inventory)

- Stok per gudang/toko (multi-lokasi)
- Kartu stok (stock card) — mutasi masuk/keluar
- Stok minimum & reorder point → alert otomatis
- Stok opname (perhitungan fisik) & penyesuaian (adjustment)
- Transfer stok antar cabang

Modul 5 — Manajemen Keuangan Standar Akuntansi (Finance)

- Chart of Accounts (COA) standar
- Jurnal umum otomatis dari event transaksi (penjualan, pembelian, barang masuk, adjustment stok)
- Buku besar (General Ledger), neraca saldo
- Laporan standar: Laba Rugi, Neraca, Arus Kas
- Piutang (AR) & Hutang (AP) supplier
- Rekonsiliasi bank (opsional integrasi open banking)

Modul 6 — Manajemen Barang Masuk (Goods Receipt)

- Penerimaan barang berdasarkan PO (matching PO vs fisik barang)
- Pengecekan kualitas & kuantitas (partial/full receipt)
- Update otomatis ke Stok & pemicu jurnal Hutang di Keuangan
- Retur ke supplier jika barang tidak sesuai

Modul 7 — Dashboard Analitik untuk Pemilik Toko

- KPI real-time: omzet harian/bulanan, margin, produk terlaris
- Analisis stok (slow-moving, stok mendekati minimum)
- Analisis keuangan (cash flow, laba rugi ringkas)
- Perbandingan performa antar cabang
- Export laporan (PDF/Excel)

---

5. Entity Relationship Diagram (ERD) — Skema Database Inti

```
erDiagram
    STORE ||--o{ USER : employs
    STORE ||--o{ WAREHOUSE : has
    USER ||--o{ ROLE : assigned

    CUSTOMER ||--o{ ORDER : places
    ORDER ||--|{ ORDER_ITEM : contains
    ORDER_ITEM }o--|| PRODUCT : refers

    PRODUCT }o--|| CATEGORY : belongs_to
    PRODUCT ||--o{ PRODUCT_VARIANT : has
    PRODUCT ||--o{ STOCK : tracked_in
    STOCK }o--|| WAREHOUSE : located_at
```

```
SUPPLIER ||--o{ PURCHASE_ORDER : receives
PURCHASE_ORDER ||--|{ PO_ITEM : contains
PO_ITEM }o--|| PRODUCT : refers

PURCHASE_ORDER ||--o{ GOODS_RECEIPT : fulfilled_by
GOODS_RECEIPT ||--|{ GR_ITEM : contains
GR_ITEM }o--|| PRODUCT : refers
GOODS_RECEIPT ||--o{ STOCK_MUTATION : triggers

ORDER ||--o{ SALE_TRANSACTION : converted_to
SALE_TRANSACTION ||--|{ SALE_ITEM : contains
SALE_TRANSACTION ||--o{ PAYMENT : paid_via
SALE_TRANSACTION ||--o{ STOCK_MUTATION : triggers

STOCK_MUTATION }o--|| PRODUCT : affects
STOCK_MUTATION }o--|| WAREHOUSE : location

SALE_TRANSACTION ||--o{ JOURNAL_ENTRY : generates
GOODS_RECEIPT ||--o{ JOURNAL_ENTRY : generates
PURCHASE_ORDER ||--o{ ACCOUNT_PAYABLE : creates
JOURNAL_ENTRY ||--|{ JOURNAL_LINE : contains
JOURNAL_LINE }o--|| ACCOUNT : posted_to

STORE {
    bigint id PK
    string name
    string address
}
USER {
    bigint id PK
    bigint store_id FK
    string name
    string email
    string role
}
CUSTOMER {
    bigint id PK
    string name
    string phone
    string email
    int loyalty_point
}
PRODUCT {
    bigint id PK
    bigint category_id FK
    string sku
    string name
    decimal price
    decimal cost_price
```

```
    int min_stock
}
PRODUCT_VARIANT {
    bigint id PK
    bigint product_id FK
    string variant_name
    string sku_variant
}
WAREHOUSE {
    bigint id PK
    bigint store_id FK
    string name
    string type
}
STOCK {
    bigint id PK
    bigint product_id FK
    bigint warehouse_id FK
    int quantity
    timestamp updated_at
}
STOCK_MUTATION {
    bigint id PK
    bigint product_id FK
    bigint warehouse_id FK
    string type
    int quantity
    string reference_type
    bigint reference_id
    timestamp created_at
}
SUPPLIER {
    bigint id PK
    string name
    string contact
    string payment_term
}
PURCHASE_ORDER {
    bigint id PK
    bigint supplier_id FK
    string status
    date po_date
    decimal total_amount
}
PO_ITEM {
    bigint id PK
    bigint po_id FK
    bigint product_id FK
    int qty_ordered
    decimal unit_price
```

```
}
GOODS_RECEIPT {
    bigint id PK
    bigint po_id FK
    bigint warehouse_id FK
    date received_date
    string status
}
GR_ITEM {
    bigint id PK
    bigint gr_id FK
    bigint product_id FK
    int qty_received
    string condition
}
ORDER {
    bigint id PK
    bigint customer_id FK
    string channel
    string status
    decimal total_amount
    timestamp created_at
}
ORDER_ITEM {
    bigint id PK
    bigint order_id FK
    bigint product_id FK
    int qty
    decimal price
}
SALE_TRANSACTION {
    bigint id PK
    bigint order_id FK
    bigint cashier_id FK
    string shift_id
    decimal total_amount
    string status
    timestamp created_at
}
SALE_ITEM {
    bigint id PK
    bigint sale_id FK
    bigint product_id FK
    int qty
    decimal price
    decimal discount
}
PAYMENT {
    bigint id PK
    bigint sale_id FK
```

```
        string method
        decimal amount
        string status
        string gateway_ref
    }
    ACCOUNT {
        bigint id PK
        string code
        string name
        string type
    }
    JOURNAL_ENTRY {
        bigint id PK
        date entry_date
        string reference_type
        bigint reference_id
        string description
    }
    JOURNAL_LINE {
        bigint id PK
        bigint journal_id FK
        bigint account_id FK
        decimal debit
        decimal credit
    }
    ACCOUNT_PAYABLE {
        bigint id PK
        bigint supplier_id FK
        bigint po_id FK
        decimal amount
        date due_date
        string status
    }
```

---

6. Use Case Diagram

```
flowchart LR
    subgraph Actors
        A1((Pelanggan))
        A2((Kasir))
        A3((Admin Gudang))
        A4((Staf Pembelian))
        A5((Finance/Akuntan))
        A6((Pemilik Toko))
        A7((Payment Gateway))
        A8((Supplier))
    end

    subgraph System["Sistem RMS"]
```

```
        UC1([Lihat Katalog & Pesan Produk])
        UC2([Cek Status Pesanan])
        UC3([Transaksi Kasir POS])
        UC4([Proses Pembayaran])
        UC5([Buka/Tutup Shift Kasir])
        UC6([Buat Purchase Order])
        UC7([Kelola Data Supplier])
        UC8([Terima Barang Masuk])
        UC9([Stok Opname])
        UC10([Transfer Stok Antar Cabang])
        UC11([Lihat Kartu Stok])
        UC12([Kelola Jurnal & COA])
        UC13([Lihat Laporan Keuangan])
        UC14([Rekonsiliasi Hutang/Piutang])
        UC15([Lihat Dashboard Analitik])
        UC16([Export Laporan])
    end

    A1 --> UC1
    A1 --> UC2
    A2 --> UC3
    A2 --> UC4
    A2 --> UC5
    UC4 --> A7
    A4 --> UC6
    A4 --> UC7
    A3 --> UC8
    A3 --> UC9
    A3 --> UC10
    A3 --> UC11
    UC8 --> A8
    A5 --> UC12
    A5 --> UC13
    A5 --> UC14
    A6 --> UC15
    A6 --> UC16
    A6 --> UC13

    UC3 -.include.-> UC4
    UC8 -.include.-> UC9
    UC4 -.extend.-> UC12
    UC8 -.extend.-> UC12
```

---

7. Class Diagram (Domain Model Inti)

```
classDiagram
    class Product {
        +Int id
        +String sku
```

```
    +String name
    +Decimal price
    +Decimal costPrice
    +Int minStock
    +getCurrentStock(warehouseId) Int
}

class Order {
    +Int id
    +String channel
    +OrderStatus status
    +Decimal totalAmount
    +addItem(product, qty)
    +checkout() SaleTransaction
    +updateStatus(status)
}

class OrderItem {
    +Int id
    +Int qty
    +Decimal price
    +subtotal() Decimal
}

class SaleTransaction {
    +Int id
    +String shiftId
    +Decimal totalAmount
    +TransactionStatus status
    +applyPayment(payment)
    +generateJournal() JournalEntry
    +reduceStock()
}

class Payment {
    +Int id
    +String method
    +Decimal amount
    +PaymentStatus status
    +confirm()
    +refund()
}

class PurchaseOrder {
    +Int id
    +POStatus status
    +Decimal totalAmount
    +approve()
    +send()
```

```
    +close()
}

class GoodsReceipt {
    +Int id
    +Date receivedDate
    +ReceiptStatus status
    +matchAgainstPO(po) Boolean
    +confirmReceipt()
    +generateStockMutation()
    +generateJournal() JournalEntry
}

class Stock {
    +Int id
    +Int quantity
    +increase(qty)
    +decrease(qty)
    +isBelowMinimum() Boolean
}

class StockMutation {
    +Int id
    +String type
    +Int quantity
    +String referenceType
}

class Supplier {
    +Int id
    +String name
    +String paymentTerm
}

class JournalEntry {
    +Int id
    +Date entryDate
    +String description
    +addLine(account, debit, credit)
    +isBalanced() Boolean
}

class JournalLine {
    +Decimal debit
    +Decimal credit
}

class Account {
    +String code
```

```
    +String name
    +AccountType type
}

class Customer {
    +Int id
    +String name
    +Int loyaltyPoint
}

Order "1" *-- "many" OrderItem
Order --> SaleTransaction : checkout
SaleTransaction "1" *-- "many" Payment
SaleTransaction --> StockMutation : triggers
SaleTransaction --> JournalEntry : generates
PurchaseOrder --> GoodsReceipt : fulfilled by
GoodsReceipt --> StockMutation : triggers
GoodsReceipt --> JournalEntry : generates
StockMutation --> Stock : updates
JournalEntry "1" *-- "many" JournalLine
JournalLine --> Account
Customer "1" --> "many" Order
Supplier "1" --> "many" PurchaseOrder
Product "1" --> "many" Stock
```

---

8. Sequence Diagram — Alur Kritis

8.1 Pemesanan Online → Konversi ke Transaksi Kasir/Pembayaran

```
sequenceDiagram
    actor Pelanggan
    participant PWA as Web PWA
    participant OrderSvc as Order Service
    participant StockSvc as Inventory Service
    participant PaySvc as Payment Service
    participant GW as Payment Gateway
    participant MQ as Event Bus (MySQL Outbox + Celery)
    participant FinSvc as Finance Service

    Pelanggan->>PWA: Pilih produk & checkout
    PWA->>OrderSvc: POST /orders
    OrderSvc->>StockSvc: Cek ketersediaan stok
    StockSvc-->>OrderSvc: Stok tersedia
    OrderSvc-->>PWA: Order dibuat (status: pending)
    PWA->>PaySvc: POST /payments
    PaySvc->>GW: Create payment request
    GW-->>Pelanggan: Redirect/QRIS untuk bayar
    Pelanggan->>GW: Konfirmasi pembayaran
    GW-->>PaySvc: Webhook payment.success
```

```
    PaySvc->>MQ: publish payment.confirmed
    MQ->>OrderSvc: consume → update status: processing
    MQ->>StockSvc: consume → decrease stock
    MQ->>FinSvc: consume → generate journal entry (AR/Kas vs Pendapatan)
    OrderSvc-->>PWA: Notifikasi status via WebSocket
    PWA-->>Pelanggan: Pesanan dikonfirmasi
```

8.2 Transaksi Kasir (Mode Offline-First PWA)

```
sequenceDiagram
    actor Kasir
    participant PWA as Kasir PWA
    participant IDB as IndexedDB (Local Queue)
    participant SW as Service Worker
    participant API as Core API (FastAPI)
    participant MQ as Event Bus

    Kasir->>PWA: Scan produk & input transaksi
    PWA->>PWA: Hitung total (logic lokal)
    alt Koneksi tersedia
        PWA->>API: POST /sales
        API->>MQ: publish sale.completed
        API-->>PWA: Transaksi sukses, cetak struk
    else Koneksi terputus
        PWA->>IDB: Simpan transaksi ke antrian lokal
        PWA-->>Kasir: Transaksi tercatat (offline), struk lokal
        SW->>SW: Background Sync terdaftar
        Note over SW,API: Saat koneksi kembali
        SW->>API: Kirim antrian transaksi (batch)
        API->>API: Validasi & deduplikasi (idempotency key)
        API->>MQ: publish sale.completed
        API-->>SW: ACK sinkron sukses
        SW->>IDB: Hapus dari antrian lokal
    end
```

8.3 Purchase Order → Barang Masuk → Update Stok & Hutang

```
sequenceDiagram
    actor Staf as Staf Pembelian
    actor Gudang as Admin Gudang
    participant ProcSvc as Procurement Service
    participant Supplier
    participant GRSvc as Goods Receipt Service
    participant StockSvc as Inventory Service
    participant FinSvc as Finance Service
    participant MQ as Event Bus

    Staf->>ProcSvc: Buat Purchase Order
    ProcSvc->>ProcSvc: Approval internal
    ProcSvc->>Supplier: Kirim PO
    Supplier-->>Gudang: Kirim barang fisik
    Gudang->>GRSvc: Input Goods Receipt (match vs PO)
```

```
    GRSvc->>GRSvc: Validasi qty & kondisi barang
    alt Sesuai PO
        GRSvc->>MQ: publish purchase.received
        MQ->>StockSvc: consume → tambah stok
        MQ->>FinSvc: consume → jurnal (Persediaan vs Hutang Usaha)
        FinSvc->>FinSvc: Buat Account Payable
        GRSvc-->>Gudang: Konfirmasi barang masuk
    else Tidak sesuai (rusak/kurang)
        GRSvc->>Gudang: Catat retur/klaim
        GRSvc->>Supplier: Ajukan retur
    end
```

8.4 Stok Opname & Penyesuaian → Jurnal Otomatis

```
sequenceDiagram
    actor Gudang as Admin Gudang
    participant StockSvc as Inventory Service
    participant MQ as Event Bus
    participant FinSvc as Finance Service

    Gudang->>StockSvc: Input hasil hitung fisik
    StockSvc->>StockSvc: Bandingkan dengan stok sistem
    alt Ada selisih
        StockSvc->>StockSvc: Buat Stock Adjustment
        StockSvc->>MQ: publish stock.adjusted
        MQ->>FinSvc: consume → jurnal selisih (Beban Susut/Persediaan)
        FinSvc-->>Gudang: Notifikasi jurnal tercatat
    else Tidak ada selisih
        StockSvc-->>Gudang: Stok sesuai, tidak ada aksi
    end
```

---

9. State Diagram — Siklus Hidup Pesanan (Order)

```
stateDiagram-v2
    [*] --> Pending: Pelanggan checkout
    Pending --> MenungguPembayaran: Order dibuat
    MenungguPembayaran --> Dibayar: Payment gateway konfirmasi
    MenungguPembayaran --> Dibatalkan: Timeout/batal
    Dibayar --> Diproses: Stok direservasi
    Diproses --> SiapDiambil: Siap ambil di toko
    Diproses --> Dikirim: Dikirim kurir
    SiapDiambil --> Selesai: Diambil pelanggan
    Dikirim --> Selesai: Diterima pelanggan
    Diproses --> Retur: Komplain/retur
    Selesai --> [*]
    Dibatalkan --> [*]
    Retur --> [*]
```

---

10. Component & Deployment Diagram

```
flowchart TB
    subgraph Client["Client Layer"]
        C1[PWA - Pelanggan]
        C2[PWA - Kasir]
        C3[Web Dashboard - Owner/Admin]
    end

    subgraph Edge["Edge/Gateway"]
        E1[Apache XAMPP + TLS]
        E2[API Gateway + Rate Limiting]
    end

    subgraph AppLayer["Application Layer - FastAPI (Python) Modular Monolith"
]
        M1[Module: Ordering]
        M2[Module: POS/Payment]
        M3[Module: Procurement]
        M4[Module: Inventory]
        M5[Module: Finance]
        M6[Module: Goods Receipt]
        M7[Module: Analytics/Reporting]
        M8[Module: Auth/RBAC]
    end

    subgraph Async["Async/Integration Layer"]
        Q1[(MySQL Outbox Event Bus)]
        W1[Worker: Celery Consumers]
    end

    subgraph Data["Data Layer"]
        D1[(MySQL XAMPP - OLTP)]
        D2[(MySQL - Cache/Session)]
        D3[(MySQL - Tabel Agregat Analytics)]
        D4[(Folder uploads - File Storage)]
    end

    subgraph External["External Services"]
        X1[Payment Gateway - Midtrans/Xendit]
        X2[SMS/WhatsApp Notification]
        X3[Supplier Email/API]
    end

    C1 & C2 & C3 --> E1 --> E2
    E2 --> M1 & M2 & M3 & M4 & M5 & M6 & M7
    M8 -.authorize.-> M1 & M2 & M3 & M4 & M5 & M6 & M7

    M1 & M2 & M3 & M6 --> Q1
    Q1 --> W1
```

```
W1 --> M4
W1 --> M5
W1 --> M7

M1 & M2 & M3 & M4 & M5 & M6 --> D1
M7 --> D3
AppLayer --> D2
AppLayer --> D4

M2 --> X1
M1 --> X2
M3 --> X3
```

---

11. Ringkasan API (Contoh Endpoint per Modul)

Modul	Endpoint Kunci
Ordering	`POST /orders`, `GET /orders/:id`, `GET /orders?customerId=`, `PATCH /orders/:id/status`
POS/Payment	`POST /sales`, `POST /sales/sync-batch` (offline sync), `POST /payments`, `POST /webhooks/payment`
Procurement	`POST /purchase-orders`, `PATCH /purchase-orders/:id/approve`, `GET /suppliers`
Goods Receipt	`POST /goods-receipts`, `POST /goods-receipts/:id/confirm`, `POST /goods-receipts/:id/return`
Inventory	`GET /stock`, `POST /stock/opname`, `POST /stock/transfer`, `GET /stock/:productId/card`
Finance	`GET /accounts`, `GET /journal-entries`, `GET /reports/profit-loss`, `GET /reports/balance-sheet`, `GET /accounts-payable`
Analytics	`GET /dashboard/kpi`, `GET /dashboard/sales-trend`, `GET /dashboard/top-products`, `GET /dashboard/export`
Auth	`POST /auth/login`, `POST /auth/refresh`, `GET /me`, RBAC middleware di semua request

---

12. Strategi PWA (Progressive Web App)

Aspek	Implementasi
Installable	Web App Manifest (icon, splash screen, `display: standalone`)
Offline Capability	Service Worker native (JavaScript) — cache-first untuk aset statis (HTML, CSS, JS), network-first untuk data produk, background sync untuk transaksi kasir offline
Data Lokal	IndexedDB (API native JavaScript) menyimpan draft order, antrian transaksi, cache produk terakhir

Aspek	Implementasi
Push Notification	Web Push API untuk status pesanan & alert stok
Responsif	Mobile-first design, breakpoint untuk tablet kasir & desktop dashboard
Idempotency	Setiap transaksi offline diberi `idempotency-key` (UUID) untuk mencegah duplikasi saat sync

---

13. Keamanan & Kepatuhan

- **Autentikasi & Otorisasi**: JWT + Refresh Token, RBAC per modul (Owner, Admin, Kasir, Gudang, Finance, Pembelian)
- **Audit Trail**: setiap perubahan data finansial & stok tercatat (siapa, kapan, apa) — tabel `audit_log`
- **Enkripsi**: TLS di transit, enkripsi kolom sensitif (data pembayaran) at-rest
- **Segregation of Duties**: approval berjenjang untuk PO & jurnal manual (mencegah fraud)
- **Data Privacy**: kepatuhan terhadap UU PDP untuk data pelanggan
- **Backup & DR**: backup otomatis MySQL (mysqldump terjadwal + binary log untuk point-in-time recovery), disaster recovery plan multi-region (opsional)

---

14. Roadmap Implementasi

Fase	Cakupan	Estimasi
Fase 1 — MVP	Modul Kasir, Stok dasar, Pemesanan online sederhana	2–3 bulan
Fase 2 — Procurement & Finance	Supplier, PO, Barang Masuk, Jurnal otomatis	2 bulan
Fase 3 — Analitik & Multi-Cabang	Dashboard, transfer stok antar cabang, laporan konsolidasi	1.5 bulan
Fase 4 — Optimasi & Skala	Migrasi modul berat (Analytics) ke service terpisah, caching lanjutan, load testing	Berkelanjutan

---

15. Pertimbangan Skalabilitas Masa Depan

Ketika volume transaksi/cabang bertambah signifikan, modul berikut adalah kandidat pertama untuk dipisah menjadi microservices independen (karena beban kerja & pola scaling berbeda dari modul lain):

1. **Analytics/Reporting** — beban query berat, cocok dipisah ke data warehouse terpisah lebih awal
2. **Inventory** — frekuensi update tinggi, kandidat sharding per cabang/wilayah

3. **Finance** — kebutuhan konsistensi tinggi & audit ketat, layak jadi service dengan database terisolasi

Boundary DDD yang sudah dirancang sejak awal membuat pemisahan ini menjadi *refactoring*, bukan *rewrite*.